

CSC3043S 2026 - Assignment 2

Title: Does the Model Know It's Hallucinating? - Probing Hidden States in Vision-Language Models

Due: 4 September 2026, 23:59 | **Total marks:** 50

Late policy: Submissions up to 24 hours late are accepted with a 10% penalty applied to the assignment mark. No submissions are accepted after that.

1. Overview and learning objectives

Vision-language models sometimes answer questions about images incorrectly - claiming an object is present when it isn't, or vice versa. This is called **object hallucination**. What's interesting is not just that it happens, but *how* it happens: does the model's internal state at the moment of answering already "know" something is off, even when the generated text and the model's own stated confidence look perfectly normal?

In this assignment you will not train or fine-tune a vision-language model. Instead, you will treat a small pretrained VLM as an object of study: you will construct a controlled question set about real images with known ground truth, run the frozen model on it, extract its internal hidden-state representations, and train a lightweight diagnostic probe to test whether those hidden states carry information about correctness that a naive confidence-based baseline misses.

By the end of this assignment you should be able to:

1. Explain what object hallucination is, how it is constructed and measured in the literature (POPE-style existence probing), and analyse why "random" and "adversarial" negative samples present different levels of difficulty.
2. Extract and correctly pool intermediate hidden-state representations from a pretrained deep multimodal model.
3. Design and run a properly controlled probing experiment - including a held-out split that never leaks into training - and choose an evaluation metric appropriate to a possibly imbalanced task.
4. Critically compare a learned probe against a simple, principled baseline, and interpret *where and why* they disagree, rather than only reporting whether one number beats another.
5. Reason about the limits of what a small-scale probing study on a self-collected dataset can and cannot support as evidence.

This is an analysis-heavy assignment: marking (Section 12) weights depth of interpretation well above volume of code. You are not expected to write a large system but rather build a small, correct pipeline and then think hard about what it tells you.

2. Background: hallucination, grounding, and why this is an interesting probing target

A VLM's *generated text* and its *stated confidence* are both downstream of whatever the model actually computed internally - but they are lossy summaries of that computation, produced by a sampling and decoding process that was never optimised to be well-calibrated. A growing line of work asks whether a model's hidden states carry usable signal about the truthfulness of its own outputs even when its surface behaviour doesn't show it (see Appendix B). This assignment asks the same question in the vision-language setting, using the cheapest experimental design that can plausibly answer it: binary existence questions with unambiguous ground truth, where "hallucination" simply means "the model's answer is wrong."

Not all wrong answers are equally informative to study. A model that incorrectly claims a dinosaur is present in a photo of a kitchen isn't a very interesting failure - that's not a *hard* case. The POPE benchmark (Li et al., 2023) formalised this by constructing negative (absent-object) questions in three ways: **random** (any absent category), **popular** (the most frequent categories overall), and **adversarial** (categories that frequently co-occur, elsewhere in the dataset, with objects that *are* present in this particular image, but happen to be absent here). Adversarial negatives are the hard case: they are exactly where a model is most likely to hallucinate from co-occurrence bias rather than from actually looking at the image. This assignment uses a simplified two-category negative construction (random and adversarial) - you are building a scaled-down version of the POPE methodology, not reusing POPE's own dataset or code.

3. Tools, model, and what you may use

3.1 The frozen model

Choose one small vision-language model, **SmolVLM-256M-Instruct** or **SmolVLM-500M-Instruct**, and use the selected model unmodified for all experiments. The SmolVLM-256M-Instruct model is recommended as it is smaller, requires fewer computational resources, and will generally result in shorter experiment times. Both models can run on a laptop or lab computer with at least 8GB of RAM, such as those available in the Senior Lab and Ishango, or on

a free-tier Colab GPU. Both models provide access to per-layer hidden states through `output_hidden_states=True`. Indicate which model you selected and use the same model throughout the assignment.

You may not fine-tune, LoRA-adapt, or otherwise update this model's weights at any point. It is a fixed object of study throughout the assignment - every experiment probes its frozen internal representations, never changes them.

3.2 Allowed and disallowed tools

Allowed. `torch`, `transformers` (to load and run the frozen model and its processor), `pycocotools` (to read the provided COCO annotations), `scikit-learn` (for the logistic-regression probe and standard metrics such as `roc_auc_score` - this is a lightweight diagnostic tool, not the object of study, so a library implementation is appropriate), `numpy`, `pandas`, `matplotlib`, `tqdm`, and the standard library. Pin your dependency versions in `requirements.txt` (e.g. `transformers>=4.46,<5`).

Not allowed. Any VLM larger than roughly 2B parameters. Any pre-existing hallucination probe, POPE evaluation script, or third-party implementation of this exact experiment - you may read the POPE paper (Appendix B) for the *idea* of random/adversarial negative construction, but the sampling, labeling, extraction, and probing code must be your own.

4. Assignment structure

The work below is split into four parts, each with a **required interface** - the function and class signatures your code must expose - and a **checkpoint**: a concrete, inspectable artefact that must exist and be verified correct before the next part makes sense to start. A checkpoint is the point at which a silent bug becomes an expensive one to trace if you skip past it. Each part's required interface takes the previous part's checkpoint artefact as its input, so the parts cannot be done out of order or skipped.

Checkpoint	Details
A – Dataset (end of §5)	<code>manifest.csv</code> exists, reproduces byte-for-byte from your stated seed, and passes every sanity check in §5.4.
B – Inference (end of §6)	A saved results file exists with one row per manifest entry, containing the generated answer, parsed label, confidence, and raw per-layer hidden states; the unclear-answer rate (Q3) and per-category accuracy (Q4) are computed from <i>this file</i> , not recomputed ad hoc.
C – Probing (end of §7)	A single train/validation index split exists and is reused, unchanged, for every layer's probe; a layer-vs-AUROC table exists before you write Q6.
D – Generalisation (end of §8)	The cross-category split (§8.1) reuses Checkpoint C's held-out examples rather than a fresh random split; your disagreement table (Q9) is generated from saved predictions, not manually inspected printouts.

5. Part A - Building a seeded question set

5.1 Source data

Use a subset of the **COCO 2017 validation set**: the **2017 Val images** archive (4,952 images, < 1GB) together with the `instances_val2017.json` annotation file (instance-segmentation annotations for those same 4,952 images, covering all 80 object categories). These two files are on Amathuba (Resources -> Datasets). The 4,952 images of the original 5,000 (48 were removed) provide a sufficiently large pool from which to sample the 200 images required for this assignment, while also ensuring that the co-occurrence statistic (Section 5.3) is computed over a reasonably representative sample of the category distribution.

5.2 Per-student seeding (note-this-before-you-start)

Use your student number as your random seed for the sampling step below:

```
student_number = "ABCDEF001"
seed = int.from_bytes(
    student_number.encode("utf-8"),
    byteorder="big"
)
rng = numpy.random.default_rng(seed=seed)
```

This means your image sample, your negative-category choices, and your exact numerical results will differ from every other student's. Your report must state your seed, and your submitted dataset manifest (Section 11) must

be reproducible from that seed by a script in your repository. A report whose numbers exactly match another student's, or whose manifest does not reproduce from the stated seed, will be considered invalid and receive a failing mark.

5.3 Constructing the question set

Sample **200 images** from the subset. For each sampled image, using the ground-truth instance annotations, construct exactly three existence questions of the form “*Is there a {object} in this image?*”:

1. **Present.** One object category confirmed present in the image by the annotations. Ground truth: yes.
2. **Absent - random.** One object category confirmed absent from the image, sampled uniformly at random from the categories not present. Ground truth: no.
3. **Absent - adversarial.** One object category confirmed absent from *this* image, but selected because it frequently co-occurs, across the dataset as a whole, with a category that *is* present in this image. You must compute this co-occurrence statistic yourself from the annotation file (e.g. for every pair of categories, how often they appear together across all images) and use it to pick the negative. Ground truth: no.

This gives $3 \times 200 = 600$ labelled question-image pairs. Save the resulting question set as a manifest (image ID, category, question text, question type, ground truth) before engaging with the model - this separates “did I build the dataset correctly” from “did I run the model correctly” as two things you can debug independently.

5.4 Sanity checks

Write these as tests: category counts are balanced across the three question types; no (image, category) pair appears twice; every “present” label is directly verifiable against the annotation file; your co-occurrence statistic, computed twice, is identical (determinism - a non-deterministic dataset construction step will make every downstream result unreproducible); your adversarial negatives are never categories that are actually present in the target image (a bug here silently corrupts your hardest and most important examples).

5.5 Required interface

Implement `src/dataset_construction.py` exposing at least:

```
def load_coco_subset(annotation_path: str, image_dir: str) -> "COCOSubset":
    """Wraps pycocotools access to the provided COCO subset."""

def compute_cooccurrence(coco: "COCOSubset") -> dict[tuple[str, str], int]:
    """Symmetric pairwise category co-occurrence counts across every image in coco."""

def sample_image_ids(coco: "COCOSubset", n_images: int, seed: int) -> list[int]:
    """Deterministic image-ID sample for the given seed. Calling this twice with the
    same seed must return identical output - this is one of your sanity checks."""

def build_question_set(coco: "COCOSubset", image_ids: list[int],
                      cooccurrence: dict[tuple[str, str], int],
                      seed: int) -> list[dict]:
    """Returns exactly 3 records per image ID: one each of question_type
    "present" / "absent_random" / "absent_adversarial", each a dict with keys
    image_id, category, question, question_type, ground_truth."""

def save_manifest(questions: list[dict], path: str) -> None: ...
def load_manifest(path: str) -> list[dict]: ...
```

`build_question_set` is the function that is expected to require the greatest degree of engagement. In particular, the adversarial-negative selection logic requires an understanding of co-occurrence bias and its implications for data construction, rather than simply implementing a generic sampling procedure.

Checkpoint A. Run your dataset-building script end to end, commit the resulting `data/manifest.csv`, and confirm every check in §5.4 passes on it before writing any inference code. If you regenerate the manifest later, the new one must reproduce identically from the same seed - if it doesn't, something in your pipeline is non-deterministic and needs fixing before you build on top of it.

5.6 Questions - Part A

Q1. Report the number of QA pairs per question type, confirm the split is balanced, and state your random seed. (A small table and a short description.)

Q2. Describe how you computed the adversarial co-occurrence statistic, and give one concrete example from your own dataset: the category present in the image, the adversarial category you chose as the negative, and the co-occurrence frequency that made you choose it. (2-3 sentences.)

6. Part B - Running the model and extracting internals

6.1 Inference

For each QA pair, prompt your chosen frozen VLM with the image and the question, generate a short answer, and record: the generated text, parsed into yes / no / unclear; the token-level probability (or log-probability) the model assigned to its generated answer token(s) - this is your confidence baseline; and the hidden states at every layer, via `output_hidden_states=True`.

NB Cache the vision-encoder output per image rather than recomputing it for all three questions about the same image, if your model's architecture makes this convenient - state whether you did this and why it matters for your runtime budget.

6.2 Deferred pooling

A layer's hidden states are a sequence of per-token vectors. **Do not** pool them down to a single vector in this part - save the raw per-layer sequence, and do the pooling in Part C, as its own explicit, justified step.

6.3 Labeling correctness

An example's **grounded** / **hallucinated** label is determined by whether the parsed generated answer matches the ground truth from Section 5.3. Decide explicitly how you handle "unclear" parses (e.g. the model didn't produce a clean yes/no) - dropping them, or mapping them to a fixed class, are both defensible, but you must state and justify your choice.

6.4 Required interface

Implement `src/inference.py` exposing at least:

```
class InferenceResult:
    image_id: int
    category: str
    question_type: str
    ground_truth: bool
    generated_text: str
    parsed_answer: bool | None          # None means "unclear"
    confidence: float                  # probability assigned to the generated answer token(s)
    hidden_states: dict[int, "np.ndarray"] # layer index -> (seq_len, hidden_dim) array,
                                           # raw, unpooled - see §6.2

def load_model(model_name: str, device: str) -> tuple["PreTrainedModel", "ProcessorMixin"]: ...

def run_single_example(model, processor, image, question: str) -> InferenceResult:
    """One (image, question) pair in, one InferenceResult out. Must not write to disk -
    that's the caller's job, so this function stays testable on a single toy example."""

def run_inference_on_manifest(model, processor, manifest: list[dict],
                              image_dir: str) -> list[InferenceResult]:
    """Iterates the manifest, caching the vision-encoder pass per image_id rather than
    per question if your architecture allows it. Returns one InferenceResult per row."""

def save_results(results: list[InferenceResult], path: str) -> None: ...
def load_results(path: str) -> list[InferenceResult]: ...
```

Checkpoint B. Before writing any probing code, run inference on a small debug subset (10-15 images, one question of each type - see Section 10), print the shape of a single example's hidden state for one layer, and confirm it's what you expect (sequence length $\times$ hidden dimension). Only then run the full manifest and save the results file. Compute the unclear-answer rate (Q3) and per-category accuracy (Q4) directly from that saved file.

6.5 Questions - Part B

Q3. Report the fraction of unclear/unparseable generations and how you handled them. (A number plus 1-2 sentences.)

Q4. Report overall answer accuracy, split by question type (present / absent-random / absent-adversarial). Which question type is hardest for the model, and does that match what you'd expect given how the categories were constructed? (A small table plus 1-2 sentences.)

Q5. State your pooling strategy (chosen when you implement Part C, but answered here since it's a property of your extraction pipeline) and justify it over at least one alternative you considered. (1-2 sentences.)

7. Part C - Probing the representations

7.1 Train/validation split

Split your dataset into a training and a held-out validation set (e.g. 70/30), stratified by question type so each split has a representative mix. **A probe must never be evaluated on data it was fit on** - this is an important experimental-hygiene requirement, mistakes here invalidate every downstream number.

7.2 The probe

For each layer (or a representative subset spanning early, middle, and late layers if running every layer is too slow), pool that layer's raw hidden states into a fixed-length vector and train a logistic-regression classifier (`sklearn.linear_model.LogisticRegression` is fine) to predict the grounded/hallucinated label from it. Report accuracy and AUROC - accuracy alone can be misleading if the classes are imbalanced, which they may well be, so check.

7.3 Required interface

Implement `src/probing.py` exposing at least:

```
def pool_layer(hidden_states: "np.ndarray", strategy: str) -> "np.ndarray":
    """(seq_len, hidden_dim) -> (hidden_dim,). strategy is one of the options you
    justified in Q5 - keep it a named parameter, not a hardcoded choice, so you can
    compare strategies later if you choose to."""

def build_feature_matrix(results: list["InferenceResult"], layer: int,
                        strategy: str) -> tuple["np.ndarray", "np.ndarray"]:
    """Returns (X, y) for one layer across all examples with a non-unclear label."""

def train_val_split(results: list["InferenceResult"], val_fraction: float,
                   seed: int) -> tuple[list[int], list[int]]:
    """Returns (train_indices, val_indices), stratified by question_type. Call this
    ONCE and reuse the same split for every layer and every probe in Parts C and D -
    a fresh random split per layer would silently leak information across your
    "generalises across layers" comparisons in Q6."""

def train_probe(X_train: "np.ndarray", y_train: "np.ndarray") -> "LogisticRegression": ...

def evaluate_probe(probe, X_val: "np.ndarray", y_val: "np.ndarray") -> dict:
    """Returns at least {"accuracy": float, "auroc": float}."""
```

Checkpoint C. Generate `train_val_split` exactly once, save the resulting indices to disk, and load them (rather than regenerating) in every subsequent script that trains a probe - including in Part D. Produce and commit a layer-vs-AUROC results table before drafting Q6, so your figure is generated from a saved artefact rather than numbers you copy in by hand.

7.4 Questions - Part C

Q6. Report probe accuracy and AUROC for every layer you tested, and plot performance against layer depth. Which region of the network (early / middle / late) is most informative about correctness? (One figure plus 1-2 sentences.)

Q7. Compare your best-layer probe's AUROC to a baseline that uses only the model's own generation confidence (e.g. thresholding the output-token probability) to predict correctness. Which does better, and by how much? (A small table plus 1-2 sentences.)

8. Part D - Generalisation and disagreement analysis

This part carries the largest share of the analysis marks. The goal is not simply to report a number, but to interpret what your specific results imply, while being honest about what your sample size can and cannot support.

8.1 Cross-category generalisation

Train a probe only on **present** and **absent-random** examples, and evaluate it on held-out **absent-adversarial** examples (and, if time allows, the reverse split too). Compare this to a probe trained and evaluated within the same category (i.e. a normal within-distribution split).

8.2 Disagreement analysis

Find concrete cases where your best probe's prediction disagrees with the confidence-baseline's prediction (§7.4, Q7). Inspect at least three such cases individually: what was the question, the image category, the ground truth, the model's stated confidence, and the probe's prediction?

8.3 Required interface

Implement `src/generalization.py` exposing at least:

```
def cross_category_split(results: list["InferenceResult"], train_types: list[str],
                        test_types: list[str],
                        val_indices: list[int]) -> tuple[list[int], list[int]]:
    """Builds a train/test split by question_type membership, restricted to indices
    already held out by Checkpoint C's train_val_split - this experiment must never
    train on an example that was in the within-category validation set."""

def compare_probe_to_baseline(probe_predictions: "np.ndarray",
                             confidence_predictions: "np.ndarray",
                             y_true: "np.ndarray",
                             metadata: list[dict]) -> "pandas.DataFrame":
    """Returns one row per validation example where the probe and the confidence
    baseline disagree, with enough metadata (image_id, category, question_type,
    ground_truth, confidence, probe_prediction) to read off Q9's examples directly -
    you should not be hand-assembling this table from printouts."""
```

Checkpoint D. Confirm `cross_category_split`'s test set is disjoint from every example any probe in Part C was trained on. Generate the disagreement table via `compare_probe_to_baseline` and select your Q9 examples from its output, rather than searching for them manually - this keeps your reported cases a fair sample rather than a cherry-picked one.

8.4 Questions - Part D

Q8. Report the generalisation gap: probe performance when trained and tested within the same category, versus trained on present/random and tested on adversarial. What does the size of this gap suggest about whether the hidden states encode a general “am I hallucinating” signal, or something more specific to how the negative examples were constructed? (A small table plus 2-3 sentences.)

Q9. Present at least three concrete disagreement examples between the probe and the confidence baseline (a small table: image/category, question, ground truth, model confidence, probe prediction). What pattern, if any, do you see? (A table plus 2-4 sentences.)

Q10. Synthesise Q6-Q9: does your evidence support the claim that this model's hidden states “know” about a hallucination before it's spoken, even when its stated confidence doesn't show it? Be explicit about what your sample size, your single model, and your specific negative-construction method do and do not let you conclude. (A short paragraph, roughly 150-250 words.)

9. Optional extension (bonus)

Attempt only if Parts A-D are complete and logged. Pick **one**:

- Repeat the pipeline with a second small VLM and compare which layers are informative and how the baseline-vs-probe gap changes.
- Replace the logistic-regression probe with a small nonlinear probe (e.g. a two-layer MLP) and discuss whether it outperforms the linear probe, and what that would or wouldn't imply about how the information is encoded.
- Extend beyond existence questions to a second question type (counting, or a simple attribute check such as colour) and discuss whether your findings hold.

Q11 (bonus). Report your chosen extension's results and a short discussion (a table plus 2-3 sentences).

10. Compute budget and access

You should develop and debug the entire pipeline on CPU (your own laptop) and/or a free-tier Colab GPU - nothing in Parts A-D requires anything more than that to get working correctly on a small debug subset. You will then be given a limited allocation of GPU time for your full-scale runs and the optional extension. Managing that allocation well, rather than burning it on avoidable recomputation, is part of what this assignment assesses.

10.1 Phases

Phase 1 - development. Build and debug the full pipeline - dataset construction, inference, pooling, probing, and the generalisation split - against a small debug subset (10-15 images, one question of each type). Confirm every checkpoint in Section 4 passes on this subset before scaling up. A free Colab T4, or a modern laptop CPU, is more than sufficient for this phase.

Phase 2 - full run. A personal laptop can run the full 600-question inference pass unattended (budget several hours, e.g. 3-5) - this is sufficient on its own, and GPU access is a convenience, not a requirement. If you use free-tier Colab instead, note that idle sessions disconnect after roughly 90 minutes, likely shorter than a full run - your inference script must save incrementally and be resumable (§6.4) before you start a long run there.

10.2 Budgeting

Time your first **20** inference calls after a short warm-up and use that to project your total runtime - do not assume the answer, since it depends on your model choice, your batching, and whether you're caching the vision encoder

correctly. Record the figure in your experiment log.

Storage: Expect on the order of **15-23GB** for the full raw hidden-state cache if saved as float16 binary arrays per §6.4; do not store this cache as JSON (see that section’s warning) or you risk several times that, and a run failing from a full disk partway through.

NB: *Things that will waste your time include recomputing the vision encoder per question instead of per image; re-running inference because you didn’t save the raw hidden states the first time; regenerating your train/validation split per layer instead of reusing one saved split; reloading the model from disk inside a loop instead of once at the start.*

11. Deliverables

Code repository (with git history). AGENTS.md present unmodified from the first commit onward. At least **8 commits**. Do not commit the COCO images; do commit your question-set manifest, cached feature files if small, and experiment logs. Suggested layout:

```
csc3043s_a2/
  AGENTS.md
  README.md
  requirements.txt
  .gitignore
  src/
    dataset_construction.py  # §5.5 interface: sampling, co-occurrence stats, manifest generation
    inference.py             # §6.4 interface: VLM loading, generation, confidence + hidden states
    probing.py               # §7.3 interface: pooling, train/val split, probes, metrics
    generalization.py        # §8.3 interface: cross-category split, disagreement table
  scripts/
    build_dataset.py         # produces data/manifest.csv           (Checkpoint A)
    run_inference.py         # produces data/inference_results.* (Checkpoint B)
    run_probes.py            # produces the split + AUROC files   (Checkpoint C)
    run_generalization.py    # produces data/disagreements.csv   (Checkpoint D)
    make_plots.py
  tests/
    test_dataset_construction.py  test_pooling.py  test_probing.py
  data/
    manifest.csv                 # Checkpoint A - your question set + labels
    train_val_split.json         # Checkpoint C - the ONE split reused everywhere
    layer_auroc.csv              # Checkpoint C
    disagreements.csv            # Checkpoint D
  logs/
  configs/
```

The `data/` files listed above are the checkpoint artefacts from Section 4 - they should exist in your repository (small text/CSV/JSON, not the raw hidden-state cache if that’s large) so a marker can see at which stage your pipeline was working, independent of your final report.

Report (PDF), at most 5 pages excluding appendices. Structured as answers Q1–Q10 (Q11 if attempted), grouped by part, with a short (≤ 1 page) introduction covering your model choice, your seed, and any non-obvious implementation decisions.

Experiment log. Every inference run and every probe you trained: configuration, dataset size, runtime, and a one-line note.

AI usage declaration, confirming AGENTS.md was in context. A declaration of “none” is perfectly acceptable.

12. Marking

Total: 50 marks, plus up to 4 bonus marks for the optional extension (Section 9). Marking deliberately weights interpretation over implementation volume - a short, well-reasoned answer that honestly states the limits of the evidence will outscore a longer answer that overclaims.

Criterion	Marks
Conceptual understanding of hallucination and grounding	7
Data construction and labeling (Part A)	8
Hidden-state extraction and probing methodology (Parts B-C)	10
Experimental design and fairness of comparisons (Part C-D)	7

Criterion	Marks
Depth of analysis and interpretation (Part D)	10
Error analysis and discussion of limitations	5
Clarity of communication and reproducibility	3
Total	50
Extension (Section 9)	bonus, up to 5

13. Academic integrity and AI usage

The Department's academic dishonesty policy and the AI usage rules in the Notes to Students apply in full. In addition, for this assignment:

AGENTS.md is mandatory. When you use an AI coding agent or a chat interface on this assignment, you must place the provided **AGENTS.md** file in the assistant's context: for agents, keep it at the repository root where the tool reads it automatically; for chat interfaces, paste or attach it at the start of every conversation. It constrains the assistant to explanation, debugging guidance, and code review, and forbids it from writing solution code or drafting your analysis.

In short: AI may explain concepts, point you to relevant documentation, ask guiding questions to help you debug, and review code you've already written to flag likely bugs or missing edge cases - without providing the fix. It may not complete any part of your dataset-construction, inference, probing, or generalisation implementation, or give you the solution or approach to a problem you haven't solved yourself.

Because your dataset and results are seeded from your own student number, a report that reproduces someone else's numbers, or a manifest that does not regenerate from your stated seed, will be treated as an integrity matter, not a coincidence.

Third-party code. You may not consult or copy from any existing implementation of this exact experiment. You may consult official library documentation, the lecture slides, and the papers cited in Appendix B.

Oral assessment. A sample of submissions may be selected for a short oral assessment in which you will be asked to explain your own code and results. Marks may be adjusted based on that assessment.

Appendix A: Suggested question template

"Is there a {object} in this image? Answer yes or no."

Keep the template fixed across all three question types so that wording itself is not a confound - you are studying the effect of *what's being asked about*, not *how the question is phrased*.

Appendix B: References

1. Li et al. (2023). Evaluating Object Hallucination in Large Vision-Language Models (POPE).
2. Kadavath et al. (2022). Language Models (Mostly) Know What They Know.
3. Azaria & Mitchell (2023). The Internal State of an LLM Knows When It's Lying.
4. Burns et al. (2022). Discovering Latent Knowledge in Language Models Without Supervision.
5. CH-Wang et al. (2024). Do Androids Know They're Only Dreaming of Electric Sheep? (internal representations and hallucination in vision-language models).
6. Lin, Lin & Belongie (2014) / Lin et al. COCO: Common Objects in Context.
7. Marafioti et al. (2025). SmolVLM: Redefining Small and Efficient Multimodal Models.